



Docker + Kubernetes en WSL2

Guía Completa para Ubuntu 24.04 en Windows WSL

Con Portainer · kind · App Web de ejemplo con múltiples réplicas

Versión	1.0 — Mayo 2025
Plataforma	Windows 10/11 + WSL2 + Ubuntu 24.04
Herramientas	Docker Engine, Portainer CE, kind, kubectl, Helm
Nivel	Principiante → Intermedio
Owner	Ing. Andrés Felipe Rojas Parra



Tabla de Contenido

1. Prerequisitos y configuración de WSL2
2. Instalación de Docker Engine
3. Instalación de Portainer (Dashboard visual)
4. Instalación de kubectly y kind (Kubernetes local)
5. Instalación del Kubernetes Dashboard
6. Proyecto completo: App web dockerizada en Kubernetes
 - 6.1 Estructura del proyecto
 - 6.2 Código de la aplicación (Flask)
 - 6.3 Dockerfile explicado línea a línea
 - 6.4 Construir y probar la imagen Docker
 - 6.5 Desplegar en Kubernetes con 3 réplicas
 - 6.6 Verificar el despliegue
 - 6.7 Escalar y actualizar
7. Resumen de comandos rápidos

1. Prerequisitos y Configuración de WSL2

⚠ IMPORTANTE: Esta guía asume Windows 10 (versión 2004+) o Windows 11 con WSL2 ya instalado. Si no lo tienes, sigue estos pasos.

1.1 ¿Qué es WSL2?

WSL2 (Windows Subsystem for Linux 2) es una capa de compatibilidad que permite ejecutar un kernel Linux real dentro de Windows. A diferencia de WSL1 (que traducía llamadas del sistema), WSL2 usa una máquina virtual ligera con el kernel Linux completo, lo que permite ejecutar Docker de forma nativa.

1.2 Activar WSL2 y Ubuntu 24.04

Abre PowerShell como Administrador y ejecuta:

```
# Instalar WSL2 con Ubuntu 24.04 (una sola línea)
wsl --install -d Ubuntu-24.04

# Verificar que WSL2 es la versión por defecto
wsl --set-default-version 2

# Ver las distribuciones instaladas
wsl --list --verbose
```

i NOTA: Después de instalar Ubuntu 24.04, se te pedirá crear un usuario y contraseña. Guárdalos, los necesitarás para los comandos sudo.

1.3 Configurar memoria y CPU para WSL2

Por defecto, WSL2 puede consumir mucha RAM. Es recomendable limitarlo. Crea el archivo `.wslconfig` en tu carpeta de usuario de Windows (`C:\Users\TuUsuario\.wslconfig`):

```
# Abre el Bloc de Notas desde PowerShell
notepad $env:USERPROFILE\.wslconfig
```

Escribe el siguiente contenido:

```
[wsl2]
memory=4GB          # Máximo 4 GB de RAM para WSL2
processors=2        # Máximo 2 núcleos de CPU
swap=2GB            # Espacio de intercambio
localhostForwarding=true # Acceder a puertos de WSL desde Windows
```

```
# Reiniciar WSL2 desde PowerShell para aplicar cambios
wsl --shutdown
# Luego abre Ubuntu 24.04 normalmente
```

1.4 Actualizar el sistema Ubuntu

Dentro de Ubuntu 24.04 (terminal WSL), ejecuta siempre esto antes de instalar cualquier cosa:

```
sudo apt update && sudo apt upgrade -y

# Instalar dependencias básicas
sudo apt install -y ca-certificates curl gnupg lsb-release apt-transport-https
git
```

1.5 Eliminar versiones antiguas de Docker

Si hay restos de instalaciones previas, elimínalos para evitar conflictos:

```
sudo apt remove -y docker.io docker-doc docker-compose docker-compose-v2
podman-docker
sudo apt autoremove -y
```

2. Instalación de Docker Engine

⚠ IMPORTANTE: Instalaremos Docker Engine directamente desde el repositorio oficial de Docker. NO uses el paquete `docker.io` de `apt`, ya que suele estar desactualizado.

2.1 Agregar la clave GPG y el repositorio oficial de Docker

Esto permite a `apt` verificar que los paquetes descargados son auténticos y vienen de Docker Inc.:

```
# Crear directorio para claves de paquetes
sudo install -m 0755 -d /etc/apt/keyrings

# Descargar e importar la clave GPG de Docker
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# Dar permisos de lectura a la clave
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Agregar el repositorio de Docker para Ubuntu Noble (24.04)
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu noble stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Actualizar la lista de paquetes
sudo apt update
```

2.2 Instalar Docker Engine y sus componentes

```
sudo apt install -y \
  docker-ce \           # Docker Engine (daemon)
  docker-ce-cli \       # Herramienta de línea de comandos (docker)
  containerd.io \       # Runtime de contenedores
  docker-buildx-plugin \ # Plugin para builds avanzados (multi-arch)
  docker-compose-plugin # Plugin docker compose (v2)
```

2.3 Iniciar y habilitar Docker

En WSL2, `systemd` puede estar desactivado en versiones antiguas. En Ubuntu 24.04 con WSL2 moderno suele funcionar bien:

```
# Habilitar Docker para que inicie automáticamente
sudo systemctl enable docker

# Iniciar el servicio ahora mismo
sudo systemctl start docker

# Verificar que está activo
sudo systemctl status docker
```

⚠ IMPORTANTE: Si `systemctl` no funciona en tu WSL2, usa: `sudo service docker start` — Esto ocurre en instalaciones de WSL2 sin `systemd` habilitado.

2.4 Agregar tu usuario al grupo docker

Sin esto, necesitarías escribir `sudo` antes de cada comando `docker`:

```
# Agregar tu usuario actual al grupo docker
sudo usermod -aG docker $USER

# IMPORTANTE: Cerrar sesión y volver a entrar para aplicar el cambio
# En WSL2 puedes hacer:
exit
# Luego vuelve a abrir Ubuntu 24.04
```

2.5 Verificar la instalación

```
# Verificar versión instalada
docker --version
# Salida esperada: Docker version 26.x.x, build xxxxxxxx

# Prueba completa: descarga y ejecuta una imagen de prueba
docker run hello-world
# Deberías ver: 'Hello from Docker!'

# Ver contenedores en ejecución
docker ps

# Ver todas las imágenes descargadas
docker images
```

3. Instalación de Portainer *(Dashboard visual de Docker)*

Portainer es una interfaz web que permite gestionar contenedores, imágenes, volúmenes y redes Docker de forma visual, sin necesidad de usar la línea de comandos para todo. Es ideal para aprender y para monitoreo.

3.1 Crear volumen persistente para Portainer

Este volumen guarda la configuración de Portainer aunque el contenedor se reinicie o elimine:

```
docker volume create portainer_data
```

3.2 Ejecutar Portainer CE (Community Edition)

```
docker run -d \
  -p 8000:8000 \           # Puerto para Edge Agent (no lo usaremos ahora)
  -p 9443:9443 \           # Puerto HTTPS de la interfaz web
  --name portainer \       # Nombre del contenedor
  --restart=always \       # Reiniciar automáticamente si el sistema se
reinicia
  -v /var/run/docker.sock:/var/run/docker.sock \ # Socket para controlar
Docker
  -v portainer_data:/data \ # Volumen persistente para configuración
  portainer/portainer-ce:latest
```

i NOTA: El parámetro `-v /var/run/docker.sock` es lo que permite a Portainer comunicarse con Docker. Es como darle a Portainer acceso directo al daemon de Docker.

3.3 Acceder a Portainer desde Windows

Gracias a `localhostForwarding=true` en `.wslconfig`, puedes abrir el navegador de Windows:

```
https://localhost:9443
# o
https://127.0.0.1:9443
```

⚠ IMPORTANTE: El navegador mostrará una advertencia de seguridad porque el certificado es autofirmado. Haz clic en 'Avanzado' y luego 'Continuar a localhost'. Esto es normal en entornos locales.

3.4 Configuración inicial de Portainer

1. Al abrir por primera vez, crea un usuario administrador (pon un nombre de usuario y contraseña fuertes).
2. Selecciona la opción 'Get Started' para gestionar el entorno Docker local.
3. Verás el dashboard con métricas: contenedores, imágenes, volúmenes, redes.

3.5 Verificar que Portainer está corriendo

```
# Ver el contenedor de Portainer
docker ps
# Deberías ver 'portainer' en la lista con estado 'Up'

# Ver los logs si hay algún problema
docker logs portainer
```


4. Instalación de kubectl y kind *(Kubernetes local)*

kind (Kubernetes IN Docker) crea clústeres de Kubernetes usando contenedores Docker como nodos. Es la forma más sencilla de tener Kubernetes en local para desarrollo y aprendizaje.

4.1 Instalar kubectl

kubectl es la herramienta de línea de comandos para interactuar con cualquier clúster de Kubernetes:

```
# Descargar la última versión estable de kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

# Dar permisos de ejecución
chmod +x kubectl

# Mover al directorio de binarios del sistema
sudo mv kubectl /usr/local/bin/kubectl

# Verificar la instalación
kubectl version --client
# Salida: Client Version: v1.xx.x
```

4.2 Instalar kind

```
# Definir la versión de kind (verifica la más reciente en
github.com/kubernetes-sigs/kind)
KIND_VERSION="v0.30.0"

# Descargar el binario
curl -Lo ./kind "https://kind.sigs.k8s.io/dl/${KIND_VERSION}/kind-$(uname) -
amd64"

# Dar permisos de ejecución
chmod +x ./kind

# Mover al PATH del sistema
sudo mv ./kind /usr/local/bin/kind

# Verificar instalación
kind version
```

4.3 Crear el clúster de Kubernetes

Este comando crea un clúster completo de Kubernetes corriendo dentro de contenedores Docker:

```
# Crear el clúster (tomará 2-5 minutos la primera vez)
kind create cluster --name dev-cluster
```

```
# Verificar que kubectl apunta al clúster recién creado
kubectl cluster-info --context kind-dev-cluster

# Ver los nodos del clúster
kubectl get nodes
# Deberías ver: dev-cluster-control-plane   Ready   control-plane   ...
```

i NOTA: Por defecto, kind crea un clúster de un solo nodo (control-plane). Para esta guía es suficiente. En la sección del proyecto veremos cómo kind distribuye múltiples Pods en ese nodo.

4.4 Comandos básicos de kubectl

Comando	Descripción
<code>kubectl get pods</code>	Lista todos los Pods del namespace actual
<code>kubectl get pods -A</code>	Lista Pods de TODOS los namespaces
<code>kubectl get nodes</code>	Lista los nodos del clúster
<code>kubectl get svc</code>	Lista los servicios (Services)
<code>kubectl get deployments</code>	Lista los Deployments
<code>kubectl describe pod <nombre></code>	Detalles completos de un Pod
<code>kubectl logs <pod></code>	Ver logs de un Pod
<code>kubectl apply -f archivo.yaml</code>	Aplicar una configuración YAML
<code>kubectl delete -f archivo.yaml</code>	Eliminar recursos de un YAML
<code>kubectl scale deployment <n> --replicas=5</code>	Escalar un Deployment

5. Instalación del Kubernetes Dashboard

El Kubernetes Dashboard es una interfaz web oficial que permite visualizar y gestionar los recursos del clúster: Pods, Deployments, Services, logs, etc.

5.1 Instalar Helm

Helm es el gestor de paquetes de Kubernetes. Usaremos Helm para instalar el Dashboard:

```
# Instalar Helm con el script oficial
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Verificar instalación
helm version
```

5.2 Instalar el Dashboard con Helm

```
# Agregar el repositorio del Dashboard
helm repo add kubernetes-dashboard https://kubernetes.github.io/dashboard/
helm repo update

# Instalar el Dashboard en su propio namespace
helm upgrade --install kubernetes-dashboard kubernetes-dashboard/kubernetes-
dashboard \
  --create-namespace \
  --namespace kubernetes-dashboard

# Verificar que los pods del dashboard están corriendo
kubectl get pods -n kubernetes-dashboard

# Espera hasta que todos muestren 'Running'
```

5.3 Crear usuario administrador

El Dashboard requiere un token de acceso. Crearemos una ServiceAccount con permisos de administrador:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ServiceAccount
metadata:
  name: admin-user
  namespace: kubernetes-dashboard
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: admin-user
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
```

```
name: admin-user
namespace: kubernetes-dashboard
EOF
```

5.4 Obtener el token de acceso

```
# Generar token (válido por defecto por 24 horas)
kubectl -n kubernetes-dashboard create token admin-user
# Copia el token generado, lo necesitarás para iniciar sesión
```

5.5 Acceder al Dashboard

```
# Hacer port-forward para acceder localmente
kubectl -n kubernetes-dashboard port-forward svc/kubernetes-dashboard-kong-
proxy 8443:443
# Este comando bloquea la terminal (déjalo corriendo en una pestaña aparte)
```

Abre en el navegador Windows:

```
https://localhost:8443
```

i NOTA: Acepta la advertencia de certificado autofirmado, selecciona 'Token' como método de login y pega el token copiado anteriormente.

6. Proyecto Completo: App Web en Docker y Kubernetes

En esta sección construiremos una aplicación web Python (Flask) desde cero, la empaquetaremos en Docker y la desplegaremos en Kubernetes con 3 réplicas para demostrar balanceo de carga y alta disponibilidad.

6.1 Arquitectura del proyecto

```

Browser (Windows)
|
localhost:30800 (NodePort Service)
|
Kubernetes Service (load balancer interno)
/   |   \
Pod-1 Pod-2 Pod-3 (3 réplicas de demo-web:1.0)

```

6.2 Estructura de archivos del proyecto

```

mi-app-k8s/
├── app/
│   ├── app.py           # Código principal de la aplicación Flask
│   ├── requirements.txt  # Dependencias Python
│   └── templates/
│       └── index.html    # Plantilla HTML de la app
├── Dockerfile           # Instrucciones para construir la imagen
└── k8s/
    ├── deployment.yaml  # Definición del Deployment (3 réplicas)
    └── service.yaml      # Definición del Service (NodePort)

```

Crea la estructura de directorios:

```

mkdir -p mi-app-k8s/app/templates mi-app-k8s/k8s
cd mi-app-k8s

```

6.3 Código de la aplicación Flask

app/app.py

Esta aplicación es un servidor web que muestra información del Pod en que está corriendo (útil para verificar el balanceo de carga entre réplicas):

```

import os
import socket
from flask import Flask, render_template

app = Flask(__name__)

# — Ruta principal —

```

```
@app.route("/")
def home():
    # Obtenemos el nombre del Pod (hostname) y la versión de la app
    pod_name = socket.gethostname() # En Kubernetes = nombre del Pod
    app_version = os.getenv("APP_VERSION", "1.0") # Variable de entorno opcional
    pod_ip = socket.gethostbyname(pod_name) # IP interna del Pod

    return render_template(
        "index.html",
        pod_name=pod_name,
        app_version=app_version,
        pod_ip=pod_ip
    )

# — Health check para Kubernetes —
@app.route("/health")
def health():
    return {"status": "ok", "pod": socket.gethostname()}, 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000, debug=False)
```

app/requirements.txt

```
Flask==3.0.3
gunicorn==22.0.0
```

✓ **TIP:** Usamos gunicorn como servidor WSGI de producción en lugar del servidor de desarrollo de Flask. Gunicorn puede manejar múltiples requests simultáneos de forma eficiente.

app/templates/index.html

Plantilla HTML que muestra la información del Pod y el estado de la aplicación:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi App en Kubernetes</title>
  <style>
    body { font-family: Arial, sans-serif; background: #1a1a2e; color: #e0e0e0;
      display: flex; justify-content: center; align-items: center;
      min-height: 100vh; margin: 0; }
    .card { background: #16213e; border: 2px solid #0f3460;
      border-radius: 12px; padding: 2rem 3rem; text-align: center; }
    h1 { color: #00b0f0; font-size: 2rem; }
    .info { background: #0f3460; border-radius: 8px; padding: 1rem;
      margin: 1rem 0; font-family: monospace; }
    .label { color: #888; font-size: 0.85rem; }
    .value { color: #00b0f0; font-size: 1.1rem; font-weight: bold; }
    .badge { background: #00b894; color: white; padding: .3rem .8rem;
      border-radius: 20px; font-size: .8rem; }
  </style>
</head>
<body>
  <div class="card">
```

```

<h1>🐳 Mi App en Kubernetes</h1>
<span class="badge">v{{ app_version }}</span>
<div class="info">
  <div class="label">Pod Name</div>
  <div class="value">{{ pod_name }}</div>
</div>
<div class="info">
  <div class="label">Pod IP</div>
  <div class="value">{{ pod_ip }}</div>
</div>
<p>Recarga la página para ver distintos Pods respondiendo 🔄</p>
</div>
</body>
</html>

```

6.4 Dockerfile explicado línea a línea

```

# — Etapa única (imagen de producción) —————

# FROM: imagen base oficial de Python 3.12 en su versión 'slim'
# 'slim' es una versión reducida que no incluye herramientas de compilación,
# por eso es más ligera (~50 MB vs ~900 MB de la versión completa)
FROM python:3.12-slim

# WORKDIR: establece el directorio de trabajo dentro del contenedor.
# Todos los comandos siguientes se ejecutarán desde /app
WORKDIR /app

# COPY requirements.txt primero (técnica de caché de capas)
# Docker cachea cada instrucción. Si solo cambia app.py pero NO
requirements.txt,
# Docker reutiliza la capa de pip install sin volver a descargar paquetes.
COPY app/requirements.txt .

# RUN: ejecuta un comando durante la construcción de la imagen
# --no-cache-dir evita guardar caché de pip (reduce tamaño de la imagen)
RUN pip install --no-cache-dir -r requirements.txt

# Ahora copiamos el resto del código de la aplicación
COPY app/ .

# EXPOSE: documenta qué puerto usa la aplicación (no abre el puerto, es
informativo)
EXPOSE 8000

# ENV: define variables de entorno disponibles en el contenedor
ENV APP_VERSION=1.0

# CMD: comando que se ejecuta al iniciar el contenedor
# gunicorn: servidor WSGI de producción
# -w 2: 2 workers (procesos) para manejar requests concurrentes
# -b 0.0.0.0:8000: escuchar en todas las interfaces en puerto 8000
# app:app → módulo 'app', objeto Flask llamado 'app'
CMD ["gunicorn", "-w", "2", "-b", "0.0.0.0:8000", "app:app"]

```

6.5 Construir y probar la imagen Docker

Construir la imagen

```
# Construir la imagen y etiquetarla como demo-web:1.0
# El punto final (.) indica que el Dockerfile está en el directorio actual
docker build -t demo-web:1.0 .

# Verificar que la imagen se creó
docker images | grep demo-web
# Verás: demo-web 1.0 <id> ... 150MB aprox
```

Probar la imagen localmente con Docker

```
# Ejecutar el contenedor
# -d: en segundo plano (detached)
# --name: nombre del contenedor
# -p 8080:8000: mapear puerto 8080 de WSL al puerto 8000 del contenedor
docker run -d --name demo-web -p 8080:8000 demo-web:1.0

# Probar desde línea de comandos
curl http://localhost:8080

# O abrir en el navegador de Windows:
# http://localhost:8080

# Ver logs del contenedor
docker logs demo-web

# Detener y eliminar el contenedor de prueba
docker stop demo-web && docker rm demo-web
```

6.6 Manifiestos de Kubernetes (YAML)

k8s/deployment.yaml — El Deployment

Un Deployment le dice a Kubernetes cuántas réplicas (Pods) quieres tener y cómo deben ser:

```
apiVersion: apps/v1          # Versión de la API para Deployments
kind: Deployment              # Tipo de recurso
metadata:
  name: demo-web              # Nombre del Deployment
  labels:
    app: demo-web             # Etiqueta para identificar este recurso
spec:
  replicas: 3                  # ← IMPORTANTE: 3 Pods corriendo simultáneamente
  selector:
    matchLabels:
      app: demo-web           # Selecciona Pods con esta etiqueta
  strategy:
    type: RollingUpdate        # Actualización progresiva (sin downtime)
    rollingUpdate:
      maxSurge: 1              # Máximo 1 Pod extra durante la actualización
      maxUnavailable: 0        # Nunca dejar un Pod sin responder
  template:                    # Plantilla para crear cada Pod
    metadata:
      labels:
        app: demo-web         # Cada Pod tendrá esta etiqueta
    spec:
      containers:
```



```

- name: demo-web      # Nombre del contenedor dentro del Pod
  image: demo-web:1.0 # Imagen a usar
  imagePullPolicy: Never # No intentar descargar (imagen local de kind)
  ports:
  - containerPort: 8000 # Puerto que expone el contenedor
  env:
  - name: APP_VERSION # Variable de entorno inyectada
    value: "1.0"
  resources:           # Límites de recursos (buena práctica)
    requests:
      memory: "64Mi" # Memoria mínima garantizada
      cpu: "100m"    # 100 milicores de CPU
    limits:
      memory: "128Mi" # Máximo de memoria
      cpu: "200m"     # Máximo de CPU
  livenessProbe:       # Kubernetes verifica si el Pod está vivo
    httpGet:
      path: /health
      port: 8000
    initialDelaySeconds: 5 # Esperar 5s antes de empezar a verificar
    periodSeconds: 10     # Verificar cada 10 segundos
  readinessProbe:      # Kubernetes verifica si el Pod puede recibir
tráfico
    httpGet:
      path: /health
      port: 8000
    initialDelaySeconds: 3
    periodSeconds: 5

```

k8s/service.yaml — El Service

Un Service expone los Pods y distribuye el tráfico entre ellos (balanceo de carga interno):

```

apiVersion: v1
kind: Service
metadata:
  name: demo-web
spec:
  type: NodePort      # Expone el servicio en un puerto del nodo
  selector:
    app: demo-web     # Selecciona todos los Pods con esta etiqueta
                    # ← Kubernetes balanceará entre los 3 Pods
  ports:
  - protocol: TCP
    port: 8000        # Puerto interno del Service
    targetPort: 8000  # Puerto del contenedor al que enviar el tráfico
    nodePort: 30800   # Puerto externo del nodo (rango válido: 30000-
32767)

```

6.7 Desplegar en Kubernetes

Paso 1: Cargar la imagen en kind

kind tiene su propio registro de imágenes. Debes cargar la imagen manualmente ya que no usa el daemon Docker directamente:

```

# Cargar la imagen local en el clúster kind
kind load docker-image demo-web:1.0 --name dev-cluster

```

```
# Mensaje esperado: Image: 'demo-web:1.0' with ID 'sha256:...' loaded
```

Paso 2: Aplicar los manifiestos YAML

```
# Aplicar el Deployment (crea los 3 Pods)
kubectl apply -f k8s/deployment.yaml

# Aplicar el Service (expone los Pods)
kubectl apply -f k8s/service.yaml

# O aplicar todo el directorio k8s/ de una vez
kubectl apply -f k8s/
```

Paso 3: Verificar el despliegue

```
# Ver el estado del Deployment
kubectl get deployment demo-web
# READY: 3/3 significa los 3 Pods están corriendo

# Ver los 3 Pods creados (cada uno tiene un nombre único)
kubectl get pods
# demo-web-7d9fb8c6b4-abc12    1/1    Running    0    30s
# demo-web-7d9fb8c6b4-def34    1/1    Running    0    30s
# demo-web-7d9fb8c6b4-ghi56    1/1    Running    0    30s

# Ver el Service y su NodePort
kubectl get svc demo-web
# NAME      TYPE      CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
# demo-web  NodePort  10.96.xxx.xxx  <none>         8000:30800/TCP   1m

# Ver detalles del Deployment
kubectl describe deployment demo-web
```

Paso 4: Acceder a la aplicación desde Windows

En kind con WSL2, el acceso es a través de port-forward o usando la IP del nodo:

```
# Opción A: Port-forward (más simple para desarrollo)
kubectl port-forward svc/demo-web 8080:8000
# Luego abre: http://localhost:8080

# Opción B: Obtener la IP del nodo kind
kubectl get nodes -o wide
# Copia la INTERNAL-IP y accede a: http://<IP>:30800

# Opción C: Usar el IP del contenedor del nodo kind
docker inspect dev-cluster-control-plane | grep IPAddress
# Accede a: http://<IP_ENCONTRADA>:30800
```

✓ **TIP:** Recarga la página varias veces con Ctrl+R. Notarás que el 'Pod Name' y 'Pod IP' cambian, demostrando que Kubernetes distribuye las peticiones entre los 3 Pods.

6.8 Escalar y actualizar el Deployment

Escalar a 5 réplicas

```
# Aumentar el número de réplicas a 5
kubectl scale deployment demo-web --replicas=5

# Verificar que los nuevos Pods se están creando
kubectl get pods -w      # -w = watch (modo en tiempo real, Ctrl+C para salir)
```

Actualizar la aplicación (Rolling Update)

Primero, modifica algo en app.py (por ejemplo cambia el título), luego:

```
# Construir nueva versión de la imagen
docker build -t demo-web:2.0 .

# Cargar la nueva versión en kind
kind load docker-image demo-web:2.0 --name dev-cluster

# Actualizar la imagen en el Deployment (Rolling Update automático)
kubectl set image deployment/demo-web demo-web=demo-web:2.0

# Monitorear el proceso de actualización
kubectl rollout status deployment/demo-web
# 'deployment "demo-web" successfully rolled out'

# Si hay algún problema, hacer rollback a la versión anterior
kubectl rollout undo deployment/demo-web
```

Ver logs de los Pods

```
# Ver logs de un Pod específico
kubectl logs <nombre-del-pod>

# Ver logs en tiempo real
kubectl logs -f <nombre-del-pod>

# Ver logs de todos los Pods con la etiqueta app=demo-web
kubectl logs -l app=demo-web --all-containers
```

7. Resumen de Comandos Rápidos

Docker — Comandos esenciales

Comando	Qué hace
<code>docker ps</code>	Lista contenedores en ejecución
<code>docker ps -a</code>	Lista TODOS los contenedores (incluso detenidos)
<code>docker images</code>	Lista imágenes descargadas/construidas
<code>docker build -t nombre:tag .</code>	Construye imagen desde Dockerfile
<code>docker run -d -p 8080:8000 nombre:tag</code>	Ejecuta contenedor en background
<code>docker stop nombre</code>	Detiene un contenedor
<code>docker rm nombre</code>	Elimina un contenedor
<code>docker rmi nombre:tag</code>	Elimina una imagen
<code>docker logs nombre</code>	Ver logs del contenedor
<code>docker exec -it nombre bash</code>	Entrar al shell del contenedor

Kubernetes — Comandos esenciales

Comando	Qué hace
<code>kubectl get pods</code>	Lista Pods
<code>kubectl get pods -A</code>	Lista Pods de todos los namespaces
<code>kubectl get svc</code>	Lista Services
<code>kubectl get deployment</code>	Lista Deployments
<code>kubectl describe pod <nombre></code>	Detalles del Pod
<code>kubectl logs <pod></code>	Logs del Pod
<code>kubectl apply -f archivo.yaml</code>	Aplica configuración
<code>kubectl delete -f archivo.yaml</code>	Elimina recursos
<code>kubectl scale deploy <n> --replicas=5</code>	Escala réplicas
<code>kubectl rollout status deploy/<n></code>	Estado del rollout
<code>kubectl rollout undo deploy/<n></code>	Rollback al estado anterior
<code>kubectl port-forward svc/<n> 8080:8000</code>	Reenvío de puerto
<code>kind create cluster --name <n></code>	Crear clúster kind

Comando	Qué hace
<code>kind delete cluster --name <n></code>	Eliminar clúster kind
<code>kind load docker-image img:tag --name <n></code>	Cargar imagen en kind

Flujo completo de trabajo

```
# 1. Construir imagen
docker build -t mi-app:1.0 .

# 2. Probar localmente con Docker
docker run -d --name test -p 8080:8000 mi-app:1.0
curl http://localhost:8080
docker stop test && docker rm test

# 3. Cargar en kind
kind load docker-image mi-app:1.0 --name dev-cluster

# 4. Desplegar en Kubernetes
kubectl apply -f k8s/

# 5. Verificar
kubectl get pods
kubectl get svc

# 6. Acceder
kubectl port-forward svc/mi-app 8080:8000
```

⚠ IMPORTANTE: Para eliminar completamente el clúster y empezar desde cero: `kind delete cluster --name dev-cluster` — Esto elimina todos los recursos de Kubernetes pero NO las imágenes Docker.